Rails has direct support for this type of interface; it adds a kind of macro route facility, called *resources*. Let's take a look at how the `config/routes.rb` file might have looked back in*Creating a Rails Application*:

```
  Depot::Application.routes.draw do
»   resources :products
  end
```

The `resources` line caused seven new routes to be added to our application. Along the way, it assumed that the application will have a controller named `ProductsController`, containing seven actions with given names.

You can take a look at the routes that were generated for us. We do this by making use of the handy `rails routes` command.

```
      Prefix Verb   URI Pattern
                    Controller#Action
    products GET    /products(.:format)
                    {:action=>"index", :controller=>"products"}
             POST   /products(.:format)
                    {:action=>"create", :controller=>"products"}
 new_product GET    /products/new(.:format)
                    {:action=>"new", :controller=>"products"}
edit_product GET    /products/:id/edit(.:format)
                    {:action=>"edit", :controller=>"products"}
     product GET    /products/:id(.:format)
                    {:action=>"show", :controller=>"products"}
             PATCH  /products/:id(.:format)
                    {:action=>"update", :controller=>"products"}
             DELETE /products/:id(.:format)
                    {:action=>"destroy", :controller=>"products"}
```

All the routes defined are spelled out in a columnar format. The lines will generally wrap on your screen; in fact, they had to be broken into two lines per route to fit on this page. The columns are (optional) route name, HTTP method, route path, and (on a separate line on this page) route requirements.

Fields in parentheses are optional parts of the path. Field names preceded by a colon are for variables into which the matching part of the path is placed